

Resumen de Preparación: Primer Parcial

Administración y Gestión de Bases de Datos | Conceptos Clave

1. Diseño Conceptual: Diagrama Entidad-Relación (DER)

El DER es una herramienta visual fundamental utilizada en la etapa de diseño de una base de datos. Sirve para modelar la información del mundo real, identificando qué datos se deben guardar y cómo interactúan entre sí antes de pasarlos a tablas físicas.

Entidades

Representan objetos, conceptos o sujetos del mundo real sobre los cuales queremos almacenar información. Ejemplos comunes: `Socios`, `Inscripciones`, `Actividades`.

Atributos

Son las propiedades o características específicas que describen a una entidad. Por ejemplo, la entidad `Socio` tiene atributos como `nombre`, `apellido` y `teléfono`.

Las Claves (Keys):

- **Clave Primaria (PK):** Es un atributo (o conjunto de atributos) cuyo valor es *único* para cada registro de la tabla. Nunca puede ser nulo o estar vacío. Permite identificar de forma unívoca cada fila.
- **Clave Foránea (FK):** Es un atributo en una tabla que hace referencia a la Clave Primaria de otra tabla. Su función principal es establecer el vínculo o relación entre ambas tablas.

2. Relaciones y Cardinalidades

Las relaciones indican cómo las entidades interactúan entre sí. La cardinalidad define cuántas instancias de una entidad se asocian con instancias de otra.

- **1:1 (Uno a Uno):** Un registro de la Entidad A se relaciona con un solo registro de la Entidad B, y viceversa.
- **1:N (Uno a Muchos):** Un registro de A puede relacionarse con muchos de B, pero un registro de B solo se relaciona con uno de A. Ejemplo: *Un socio realiza muchas inscripciones, pero una inscripción pertenece a un solo socio.*

Atención con relaciones N:M (Muchos a Muchos):

En un modelo relacional, las bases de datos no soportan relaciones N:M directas. ¿Cómo se soluciona? Se debe romper esa relación creando una *tabla intermedia* (o entidad asociativa) que contenga las claves foráneas de ambas tablas, transformando la relación N:M en dos relaciones 1:N.

3. Proceso de Normalización

La normalización es el proceso de estructurar y organizar los datos en tablas para evitar la redundancia de información y proteger la integridad de los datos, eliminando dependencias anómalas.

Primera Forma Normal (1FN) - Atomicidad

Exige que cada celda de la tabla contenga un valor único y atómico. Una columna no se puede subdividir (ej. no tener varios teléfonos en una sola celda separados por comas) y no deben existir grupos repetidos de columnas.

Segunda Forma Normal (2FN) - Sin Dependencias Parciales

La tabla debe estar en 1FN. Además, se deben eliminar las dependencias parciales. Esto ocurre cuando tienes una Clave Primaria compuesta (ej. `id_pedido + id_producto`) y algún atributo (ej. `nombre_producto`) depende solo de una parte de esa clave (del `id_producto`). Para solucionarlo, se separan en tablas distintas.

Tercera Forma Normal (3FN) - Sin Dependencias Transitivas

La tabla debe estar en 2FN. No debe haber dependencias transitivas; esto significa que todos los atributos no clave deben depender *única y exclusivamente* de la Clave Primaria, y no depender de otro atributo que no sea clave.

4. Introducción a SQL

SQL (Structured Query Language) es el lenguaje estándar para interactuar con bases de datos relacionales. Sus comandos se agrupan en tres grandes familias:

DDL (Data Definition)

Define estructuras (tablas, BD).

Ejemplos: `CREATE`, `ALTER`, `DROP`,
`TRUNCATE`

DML (Data Manipulation)

Opera sobre los datos mismos.

Ejemplos: `SELECT`, `INSERT`, `UPDATE`,
`DELETE`

5. Tipos de Datos Comunes en SQL

Al crear una tabla (DDL), se debe asignar un tipo de dato a cada columna según la naturaleza de la información que almacenará.

Tipo de Dato	Uso / Justificación	Ejemplo Práctico
<code>VARCHAR(n)</code>	Almacena cadenas de texto de longitud variable. El número 'n' define el límite máximo de caracteres.	Nombre, apellido, dirección, correo electrónico.
<code>DATE</code>	Formato nativo para fechas, optimizado para operaciones cronológicas (sin incluir horas).	Fecha de nacimiento, fecha de inscripción.
<code>DECIMAL(p, s)</code>	Valores numéricos exactos con decimales. 'p' es el total de dígitos y 's' cuántos son decimales.	Precio de un producto, salario, costo mensual.
<code>INT</code>	Números enteros sin decimales. Ocupa poco espacio y es rápido para cálculos.	Cantidad de unidades, códigos numéricos, edad.
<code>BOOLEAN / BIT</code>	Estados lógicos binarios (Verdadero o Falso). Máxima eficiencia en almacenamiento.	¿Es activo?, ¿Tiene deuda? (Sí/No).

6. Ejemplo Práctico: Creación de Tablas (DDL)

La sentencia `CREATE TABLE` se utiliza para crear la estructura de una nueva tabla, definiendo sus columnas, tipos de datos y restricciones (como la clave primaria).

```
-- Ejemplo de creación de la tabla Alumnos
CREATE TABLE alumnos (
  id_alumno INT PRIMARY KEY,
  nombre VARCHAR(50),
  apellido VARCHAR(50),
  fecha_nacimiento DATE
);
```

Tip para el examen: Lee siempre detenidamente los requerimientos. Identifica primero los sustantivos (serán tus tablas/entidades) y los adjetivos/datos específicos (serán tus atributos). Las acciones entre ellos (gestiona, tiene, realiza) definirán tus relaciones y claves foráneas.